

Оптимизация PostgreSQL для высоконагруженных систем

Цель

Настроить сервер PostgreSQL для максимальной производительности, оптимизировать параметры ядра Linux и СУБД под высокие нагрузки.

Требования к читателю

1. Базовые знания Linux (администрирование, работа с консолью: df, du, top, iotop, iftop)
2. Опыт работы с PostgreSQL (pg_hba, pg_stat_statements, auto_explain)
3. Доступ к серверу с правами root

Требования к инфраструктуре

1. ОС: Linux (RHEL/CentOS, Debian/Ubuntu)
2. PostgreSQL: версия 11+
3. Железо:
 - RAM: ≥ 8 ГБ (рекомендуется 32+ ГБ)
 - CPU: ≥ 4 ядра
4. Диски: SSD/NVMe (для узлов БД)
5. Доп. ПО:
 - pgbouncer (для пулинга соединений)
 - tuned (для оптимизации ядра)

Описание назначения компонента

PostgreSQL - объектно-реляционная СУБД с открытым исходным кодом, широко используемая для критически важных рабочих нагрузок.

Tuned — утилита для автоматической настройки параметров ядра Linux, оптимизирующая систему под конкретные нагрузки (СУБД, веб-серверы, виртуализацию).

PgBouncer — легковесный пулер соединений, снижающий нагрузку на PostgreSQL за счёт переиспользования подключений.

Способ установки

Установка PostgreSQL

Для установки PostgreSQL добавьте официальный репозиторий:

```
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'
wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo apt-key add -
sudo apt update
```

Далее установите PostgreSQL:

```
sudo apt install postgresql-15
```

PostgreSQL " PostgreSQL".
PGBouncer

Установите PGbouncer на узел с PostgreSQL, используя следующую команду:

```
sudo apt-get install pgbouncer -y
```

PGbouncer предназначен для управления пулом соединений к PostgreSQL и позволяет минимизировать издержки, связанные с установлением новых подключений к PostgreSQL.

По необходимости про установку и настройку PGbouncer читайте в статье "[Установка PGbouncer](#)".

Установка и настройка Tuned

```
sudo apt install tuned -y
```

Запуск и включение в автозагрузку

```
sudo systemctl enable --now tuned
```

Настройка

Для настройки PostgreSQL используйте следующие команды:

1. Создайте отдельного пользователя `elma365` с паролем `SecretPassword` для работы с БД. Имя пользователя и пароль приведены для примера:

```
sudo -u postgres psql -c "CREATE ROLE elma365 WITH login password 'SecretPassword';"
```

2. Создайте отдельную базу данных elma365 и назначьте выделенного пользователя elma365 её владельцем и кодировку `ru\_RU.UTF-8`. Нельзя использовать существующую базу, т. к. система создаёт таблицы и схемы:

```
sudo -u postgres psql -c "CREATE DATABASE elma365 WITH OWNER elma365 LOCALE_PROVIDER=icu ICU_LOCALE='ru-RU' ENCODING = 'UTF8' LC_COLLATE='ru_RU.UTF-8' LC_CTYPE='ru_RU.UTF-8' TEMPLATE = template0;"
```

3. Добавьте необходимые расширения для базы данных elma365:

```
sudo -u postgres psql -d elma365 -c "CREATE EXTENSION \"uuid-osspl\"; CREATE EXTENSION pg_trgm;"
```

4. Откройте для редактирования конфигурационный файл /etc/postgresql/13/main/postgresql.conf

```
sudo nano /etc/postgresql/15/main/postgresql.conf
```

5. Увеличьте максимальное количество подключений, изменив значение параметра max_connections на 2000:

```
shared_buffers = 8GB # 25% от RAM (если 32 ГБ)
effective_cache_size = 16GB # 50% от RAM
work_mem = 64MB # Память на сортировку/хеш-соединения
maintenance_work_mem = 1GB # Память для VACUUM, CREATE INDEX
max_connections = 2000 # Для PgBouncer лучше 2000 + пулинг
max_locks_per_transaction = 512 # Для увеличения максимального количества блокировок на транзакцию.
wal_level = replica # Для репликации
synchronous_commit = off # Увеличивает производительность (риск потери данных при сбое)
max_wal_senders = 10 # Для репликации
random_page_cost = 1.1 # Для SSD/NVMe
effective_io_concurrency = 200 # Для NVMe
```

По необходимости используйте онлайн генераторы конфигурационных параметров. Подробнее смотрите в сервисе [CYBERTEC](#).

Настройка PgBouncer

Конфигурация (/etc/pgbouncer/pgbouncer.ini):

```
[databases]
postgres = host=127.0.0.1 port=5432 dbname=elma365
* = host=127.0.0.1 port=5432

[pgbouncer]
logfile = /var/log/postgresql/pgbouncer.log
pidfile = /var/run/postgresql/pgbouncer.pid
listen_addr = *
listen_port = 6432
unix_socket_dir = /var/run/postgresql
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt
admin_users = postgres
ignore_startup_parameters = extra_float_digits,geqo
pool_mode = session
server_reset_query = DISCARD ALL
max_client_conn = 10000
default_pool_size = 800
reserve_pool_size = 150
reserve_pool_timeout = 1
max_db_connections = 1000
pkt_buf = 8192
listen_backlog = 4096
log_connections = 0
log_disconnections = 0
```

По необходимости про установку и настройку PgBouncer читайте в статье "[Установка PgBouncer](#)".

Тюнинг

Создание кастомного профиля для PostgreSQL в программе Tuned.

```
sudo mkdir -p /etc/tuned/postgresql
sudo nano /etc/tuned/postgresql/tuned.conf
```

Конфиг:

```
[main]
summary=Optimize for PostgreSQL
include=throughput-performance

[cpu]
governor=performance
energy_perf_bias=performance

[vm]
dirty_ratio=10
dirty_background_ratio=3
swappiness=1

[disk]
elevator=deadline
readahead=4096

[sysctl]
kernel.sched_min_granularity_ns = 10000000
kernel.sched_wakeup_granularity_ns = 15000000
vm.dirty_writeback_centisecs = 6000
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
```

Активируем профиль:

```
sudo tuned-adm profile postgresql
```

Проверка

```
auto_explain - PostgreSQL,
postgresql.conf, « » . ' auto_explain, postgresql.conf:
```

```
Включение auto_explain
session_preload_libraries = 'auto_explain'
auto_explain.log_min_duration = 1000; ## будут записываться запросы, выполняющиеся более 1000 миллисекунд
auto_explain.log_analyze = true; ## будет записываться вывод команды EXPLAIN ANALYZE
auto_explain.log_buffers = true; ## включение статистики об использовании буферов
```

После сбора долгих запросов необходимо выполнить их анализ.

Пример плана:

```

Query Text: SELECT (main.body) FROM "head"."tasks" AS main WHERE ("head".f_cast_isots(main.body#>>'__deletedAt') IS
NULL AND (((main.body#>>'__item')::jsonb) = '{"namespace":"service_desk","code":"night_request_handling","id":"018b822c-
e7b0-c751-1914-2b698ad2b01d"}'::jsonb) AND ((main.body#>>'__parentId')::text) IS NULL) ORDER BY "head".f_cast_isots
(main.body#>>'__createdAt') DESC, (main.body#>>'__id') LIMIT 11 OFFSET 0
Limit (cost=20249.14..235800.34 rows=11 width=1504) (actual rows=3 loops=1)
Output: body, (head.f_cast_isots((body #>>'__createdAt')::text[])), ((body #>>'__id')::text[])
Buffers: shared hit=1066102 read=249864 written=1207
I/O Timings: read=1293.051 write=17.127
-> Incremental Sort (cost=20249.14..627711.61 rows=31 width=1504) (actual rows=3 loops=1)
Output: body, (head.f_cast_isots((body #>>'__createdAt')::text[])), ((body #>>'__id')::text[])
Sort Key: (head.f_cast_isots((main.body #>>'__createdAt')::text[])) DESC, ((main.body #>>'__id')::text[])
Presorted Key: (head.f_cast_isots((main.body #>>'__createdAt')::text[]))
Full-sort Groups: 1 Sort Method: quicksort Average Memory: 31kB Peak Memory: 31kB
Buffers: shared hit=1066102 read=249864 written=1207
I/O Timings: read=1293.051 write=17.127
-> Index Scan Backward using "tasks:f__createdAt" on head.tasks main (cost=0.43..627710.21 rows=31 width=1504)
(actual rows=3 loops=1)
Output: body, head.f_cast_isots((body #>>'__createdAt')::text[]), (body #>>'__id')::text[]
Filter: (((main.body #>>'__parentId')::text[]) IS NULL) AND (((main.body #>>'__item')::text[])::jsonb = '{"id":
"018b822c-e7b0-c751-1914-2b698ad2b01d", "code": "night_request_handling", "namespace": "service_desk"}'::jsonb) AND (head.
f_cast_isots((main.body #>>'__deletedAt')::text[])) IS NULL))
Rows Removed by Filter: 633258
Buffers: shared hit=1066102 read=249864 written=1207
I/O Timings: read=1293.051 write=17.127
JIT:
Functions: 6
Options: Inlining false, Optimization false, Expressions true, Deforming true

```

Это план запроса PostgreSQL, описывающий выполнение SQL-запроса. Давайте разберем различные части плана запроса:

Query Text: это анализируемый SQL-запрос. Он выбирает данные из таблицы с именем «tasks» в схеме «head» с определенными условиями и упорядочивает результаты.

Limit : в этом разделе представлена информация об ожидаемой стоимости запроса и количестве строк, которые он ожидает вернуть. В этом случае он ожидает вернуть 11 строк.

Output: в этом разделе перечислены столбцы, которые будут включены в выходные данные запроса: «body», вычисляемое выражение и другое вычисляемое выражение.

Buffers: в этом разделе показано использование буферов, указывающее, сколько операций чтения и записи произошло. Он определяет общие обращения, чтения и записи.

I/O Timings: эта часть предоставляет информацию о времени, затраченном на операции чтения и записи.

Incremental Sort: в этом разделе представлена операция сортировки, которая является частью выполнения запроса. Он определяет критерии сортировки и использование памяти.

Index Scan Backward using: это основная операция при выполнении запроса. Это сканирование индекса «tasks: f__createdAt», которое используется для фильтрации и извлечения строк из таблицы «tasks» на основе заданных условий. Условия перечислены в разделе «Фильтр», где проверяется наличие определенных значений в данных JSONB и отсутствие значения в поле «__parentId». «Строки, удаленные фильтром» указывают, сколько строк было отфильтровано по этим условиям.

JIT: в этом разделе представлена информация о JIT-компиляции функций и настройках их оптимизации.

В целом в этом плане запроса описываются шаги, предпринятые для выполнения SQL-запроса, включая сортировку и фильтрацию, а также использование ресурсов во время выполнения. Это важно для оптимизации производительности запросов и понимания того, как ядро базы данных обрабатывает запрос.

Визуализацию планов можно посмотреть в pgAdmin или с использованием сторонних сервисов. Примеры таких сервисов: <https://explain.tensor.ru/> и <https://explain.dalibo.com/>. Эти инструменты позволяют разработчикам проанализировать выполнение SQL-запроса и получить подробную информацию о том, как PostgreSQL выполняет запрос, а также о возможных проблемах производительности.

Согласно рекомендации сервиса [Explain PostgreSQL](#) необходимо добавить индекс для увеличения быстродействия данного запроса.

```

-- (((body #>>'__item')::text[])::jsonb) :: jsonb(=)
CREATE INDEX CONCURRENTLY "~tasks-f3834152"
ON head.tasks(((body #>>'__item')::text[])::jsonb))
WHERE
(body #>>'__parentId')::text[] IS NULL;

```

После добавления индекса, проверьте выполнение данного запроса, время выполнения должно уменьшится.

Вывод

PostgreSQL + PgBouncer + Tuned создает сбалансированную систему, где:

- PostgreSQL обрабатывает данные и сложные запросы.
- PgBouncer минимизирует накладные расходы на соединения.
- Tuned обеспечивает оптимальную работу ОС для СУБД.